

Assignment 02 Extra

Digital RTL Design

Combinational & Sequential Circuit Design

Marwan Abdelfattah

Contents

1	Question 1	2
1.1	Parameterized Gray and One-Hot Encoder	2
1.1.1	Gray Encoding	2
1.1.2	One-Hot Encoding	2
1.1.3	Generate Conditional Construct	2
1.1.4	Testbenches	2
1.2	RTL Design	3
1.3	Testbenches	4
1.3.1	Gray Encoding Testbench	4
1.3.2	One-Hot Encoding Testbench	5
1.4	Simulation Waveform	6
1.4.1	Gray Simulation	6
1.4.2	One-Hot Simulation	6
2	Question 2	7
2.1	1-to-4 Demultiplexer	7
2.2	RTL Design	7
2.3	Testbench	8
2.4	Simulation Waveform	9

1 Question 1

1.1 Parameterized Gray and One-Hot Encoder

Design a Verilog module using a **generate conditional construct** that takes a 3-bit binary input $A[2:0]$ and produces a corresponding 7-bit encoded output $B[6:0]$.

The encoding method should be selected using a parameter called `USE_GRAY`, which can be set to either 1 or 0. The default value of `USE_GRAY` should be 1.

- **Input:**
 - $A[2:0]$: 3-bit binary input
- **Output:**
 - $B[6:0]$: 7-bit encoded output
- **Parameter:**
 - `USE_GRAY`: Selects the encoding method.
 - Default value: `USE_GRAY = 1`

1.1.1 Gray Encoding

If `USE_GRAY` is set to 1, the module should perform **Gray encoding** on the input. The Gray encoding logic should be implemented using an `always` block.

Gray code is a binary numeral system in which consecutive values differ by only one bit. This property reduces the possibility of errors during transitions.

1.1.2 One-Hot Encoding

If `USE_GRAY` is set to 0, the module should perform **one-hot encoding** on the input using an `always` block.

One-hot encoding represents data using a binary string in which only a single bit is set to 1, while all other bits are set to 0.

1.1.3 Generate Conditional Construct

The appropriate encoding logic should be selected at elaboration time using a **generate conditional construct** based on the value of `USE_GRAY`.

1.1.4 Testbenches

Write two separate testbenches to verify the functionality of the design for both parameter values.

1. Gray Encoding Testbench

- Instantiate the design with `USE_GRAY = 1`.
- Verify the Gray encoding functionality.
- The testbench should be **exhaustive**, testing all possible values of $A[2:0]$.

- Check the results using the simulation waveforms.

2. One-Hot Encoding Testbench

- Instantiate the design with `USE_GRAY = 0`.
- Verify the one-hot encoding functionality.
- The testbench should be **exhaustive**, testing all possible values of `A[2:0]`.
- Check the results using the simulation waveforms.

Both testbenches should verify all $2^3 = 8$ possible input combinations.

1.2 RTL Design

```

1 module encoder (A, B);
2 parameter USE_GRAY = 1;
3 input [2:0] A;
4 output reg [6:0] B;
5
6 generate
7     if(USE_GRAY) begin
8         always @(A) begin
9             case (A)
10                3'b000: B = 3'b000;
11                3'b001: B = 3'b001;
12                3'b010: B = 3'b011;
13                3'b011: B = 3'b010;
14                3'b100: B = 3'b110;
15                3'b101: B = 3'b111;
16                3'b110: B = 3'b101;
17                3'b111: B = 3'b100;
18                default: B = 3'b000;
19            endcase
20        end
21    end else
22        always @(A) begin
23            case (A)
24                3'b000: B = 7'b0000000;
25                3'b001: B = 7'b0000001;
26                3'b010: B = 7'b0000010;
27                3'b011: B = 7'b0000100;
28                3'b100: B = 7'b0001000;
29                3'b101: B = 7'b0010000;
30                3'b110: B = 7'b0100000;
31                3'b111: B = 7'b1000000;
32                default: B = 7'b0000000;
33            endcase
34        end
35    endgenerate
36 endmodule

```

1.3 Testbenches

Two separate testbenches are used to verify the functionality of the design for both possible values of the `USE_GRAY` parameter. Since the input `A` is 3 bits wide, there are $2^3 = 8$ possible input combinations. Therefore, both testbenches are exhaustive and test all possible input values.

1.3.1 Gray Encoding Testbench

The first testbench verifies the design with `USE_GRAY = 1`. The parameter is overridden during module instantiation, and all eight possible values of the 3-bit input `A` are tested.

```

1 module encoder_gray_tb();
2     reg [2:0] A;
3     reg [6:0] B_expected;
4     wire [6:0] B_dut;
5
6     encoder DUT(A, B_dut);
7
8     integer i;
9     initial begin
10         for(i=0; i<8; i=i+1) begin
11             A = i;
12             case (A)
13                 3'b000: B_expected = 3'b000;
14                 3'b001: B_expected = 3'b001;
15                 3'b010: B_expected = 3'b011;
16                 3'b011: B_expected = 3'b010;
17                 3'b100: B_expected = 3'b110;
18                 3'b101: B_expected = 3'b111;
19                 3'b110: B_expected = 3'b101;
20                 3'b111: B_expected = 3'b100;
21                 default: B_expected = 3'b000;
22             endcase
23             #10
24             if(B_dut != B_expected) begin
25                 $display("Error");
26                 $stop;
27             end
28         end
29         $stop;
30     end
31
32     initial begin
33         $monitor("A=%b, B_expected=%b, B_dut=%b",
34                 A, B_expected, B_dut);
35     end
36 endmodule

```

1.3.2 One-Hot Encoding Testbench

The second testbench verifies the design with `USE_GRAY = 0`. The parameter is overridden during module instantiation, and all eight possible values of the 3-bit input `A` are tested.

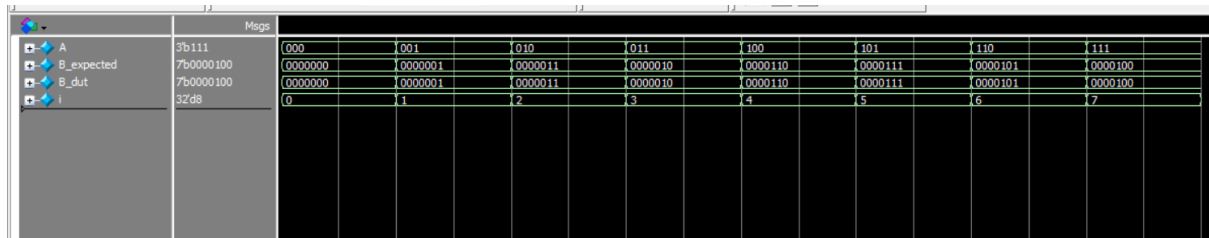
```

1 module encoder_one_hot_tb();
2     reg [2:0] A;
3     reg [6:0] B_expected;
4     wire [6:0] B_dut;
5
6
7
8     encoder #(0) DUT(A, B_dut);
9
10    integer i;
11    initial begin
12        for(i=0; i<8; i=i+1) begin
13            A = i;
14            case (A)
15                3'b000: B_expected = 7'b0000000;
16                3'b001: B_expected = 7'b0000001;
17                3'b010: B_expected = 7'b0000010;
18                3'b011: B_expected = 7'b0000100;
19                3'b100: B_expected = 7'b0001000;
20                3'b101: B_expected = 7'b0010000;
21                3'b110: B_expected = 7'b0100000;
22                3'b111: B_expected = 7'b1000000;
23                default: B_expected = 7'b0000000;
24            endcase
25            #10
26            if(B_dut != B_expected) begin
27                $display("Error");
28                $stop;
29            end
30        end
31        $stop;
32    end
33
34    initial begin
35        $monitor("A=%b, B_expected=%b, B_dut=%b",
36                A, B_expected, B_dut);
37    end
38 endmodule

```

1.4 Simulation Waveform

1.4.1 Gray Simulation



The waveform shows the simulation of a Gray encoder. The left pane lists signals: A (3b111), B_expected (7b0000100), B_dut (7b0000100), and i (32d8). The main pane displays the data for these signals across 8 time steps (0 to 7). The values for A, B_expected, and B_dut are identical for each step, representing the Gray code for the input A.

Time	A	B_expected	B_dut
0	000	0000000	0000000
1	001	0000001	0000001
2	010	0000011	0000011
3	011	0000010	0000010
4	100	0000110	0000110
5	101	0000111	0000111
6	110	0000101	0000101
7	111	0000100	0000100

Figure 1: Gray Encoding Simulation

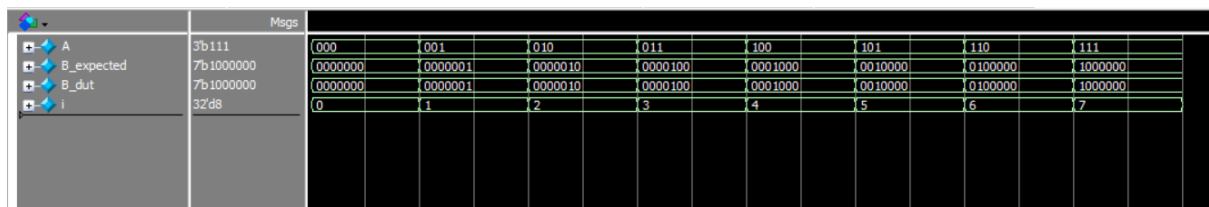
```

# A=000, B_expected=0000000, B_dut=0000000
# A=001, B_expected=0000001, B_dut=0000001
# A=010, B_expected=0000011, B_dut=0000011
# A=011, B_expected=0000010, B_dut=0000010
# A=100, B_expected=0000110, B_dut=0000110
# A=101, B_expected=0000111, B_dut=0000111
# A=110, B_expected=0000101, B_dut=0000101
# A=111, B_expected=0000100, B_dut=0000100
** Note: $stop : encoder_gray_tb.v(29)

```

Figure 2: Gray Encoding Transcript

1.4.2 One-Hot Simulation



The waveform shows the simulation of a One-Hot encoder. The left pane lists signals: A (3b111), B_expected (7b1000000), B_dut (7b1000000), and i (32d8). The main pane displays the data for these signals across 8 time steps (0 to 7). The values for A, B_expected, and B_dut are identical for each step, representing the One-Hot code for the input A.

Time	A	B_expected	B_dut
0	000	0000000	0000000
1	001	0000001	0000001
2	010	0000010	0000010
3	011	0000100	0000100
4	100	0001000	0001000
5	101	0010000	0010000
6	110	0100000	0100000
7	111	1000000	1000000

Figure 3: One-Hot Simulation

```

# A=000, B_expected=0000000, B_dut=0000000
# A=001, B_expected=0000001, B_dut=0000001
# A=010, B_expected=0000010, B_dut=0000010
# A=011, B_expected=0000100, B_dut=0000100
# A=100, B_expected=0001000, B_dut=0001000
# A=101, B_expected=0010000, B_dut=0010000
# A=110, B_expected=0100000, B_dut=0100000
# A=111, B_expected=1000000, B_dut=1000000
# ** Note: $stop : encoder_one_hot_tb.v(31)
# Time: 80 ns Iteration: 0 Instance: /encoder_one_hot_tb

```

Figure 4: One-Hot Transcript

2 Question 2

2.1 1-to-4 Demultiplexer

Design a **1-to-4 demultiplexer (Demux)** using Verilog. The demultiplexer should have a single input D and two select inputs S[1:0]. The output should consist of four signals Y[3:0], where the input signal D is routed to one of the four outputs based on the values of the select inputs.

- **Input:**

- D: 1-bit data input
- S[1:0]: 2-bit select input

- **Output:**

- Y[3:0]: 4-bit output

The select inputs determine which output receives the input signal D. Only the selected output should reflect the value of D, while all other outputs should remain LOW.

Write the Verilog code for the 1-to-4 demultiplexer and verify its functionality using an **exhaustive self-checking testbench**. The testbench should test all possible combinations of the data input D and select inputs S[1:0] and automatically compare the actual output with the expected output.

2.2 RTL Design

```
1 module demux (  
2     input D,  
3     input [1:0] S,  
4     output reg [3:0] Y  
5 );  
6 always @(*) begin  
7     case (S)  
8         2'b00: Y = {3'b000,D};  
9         2'b01: Y = {2'b00,D, 1'b0};  
10        2'b10: Y = {1'b0,D,2'b00};  
11        2'b11: Y = {D,3'b000};  
12    endcase  
13 end  
14 endmodule
```

2.3 Testbench

```
1 module demux_tb ();
2     reg D;
3     reg [1:0] S;
4     reg [3:0] Y_exp;
5     wire [3:0] Y_dut;
6
7
8     demux DUT(.D(D),.S(S),.Y(Y_dut));
9
10    integer i,j;
11    initial begin
12        for(i=0;i<4;i=i+1) begin
13            S = i;
14            for(j=0;j<2;j=j+1)begin
15                D = j;
16                case (S)
17                    2'b00: Y_exp = {3'b000,D};
18                    2'b01: Y_exp = {2'b00,D, 1'b0};
19                    2'b10: Y_exp = {1'b0,D,2'b00};
20                    2'b11: Y_exp = {D,3'b000};
21                endcase
22                #10;
23                if(Y_dut != Y_exp) begin
24                    $display("Error");
25                    $stop;
26                end
27            end
28        end
29        $stop;
30    end
31
32    initial begin
33        $monitor("D=%b, S=%b, Y_exp=%b, Y_dut=%b",
34            D, S, Y_exp, Y_dut);
35    end
36 endmodule
37
```

2.4 Simulation Waveform

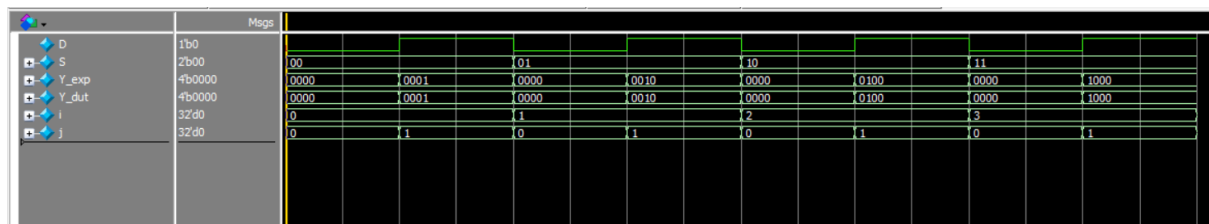


Figure 5: Simulation waveform

```

D=0, S=00, Y_exp=0000, Y_dut=0000
D=1, S=00, Y_exp=0001, Y_dut=0001
D=0, S=01, Y_exp=0000, Y_dut=0000
D=1, S=01, Y_exp=0010, Y_dut=0010
D=0, S=10, Y_exp=0000, Y_dut=0000
D=1, S=10, Y_exp=0100, Y_dut=0100
D=0, S=11, Y_exp=0000, Y_dut=0000
D=1, S=11, Y_exp=1000, Y_dut=1000
** Note: $stop      : demux_tb.v(29)
Time: 80 ns  Iteration: 0  Instance: /demux_tb

```

Figure 6: Transcript